

High-Level Design (HLD)

1. System Overview

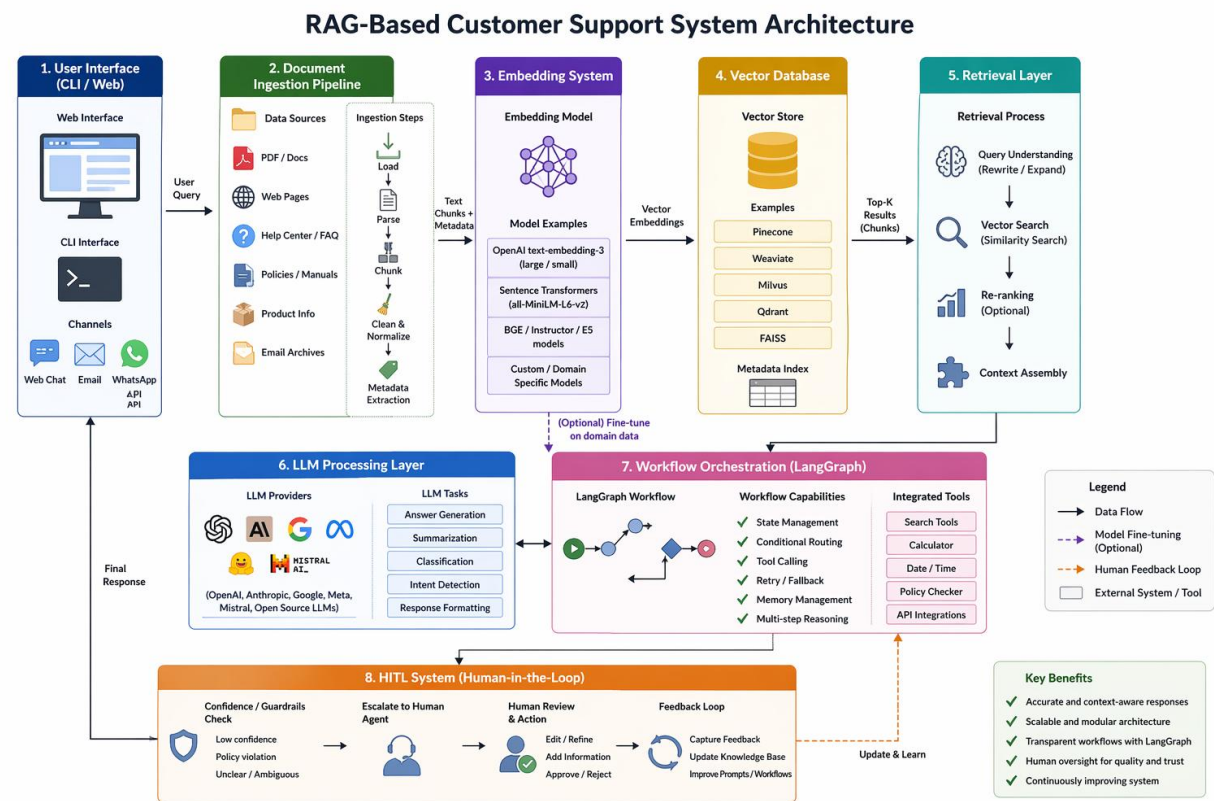
Problem Definition

Modern customer support faces a "Scalability vs. Accuracy" dilemma. Human agents are accurate but slow and expensive; traditional chatbots are fast but often "hallucinate" incorrect information or fail to recognize when a customer is frustrated or making a legal threat.

Scope of the System

This system implements an intelligent intermediary. It uses **Retrieval-Augmented Generation (RAG)** to ground AI responses in verified PDF documents and employs a **Graph-based workflow** to detect user intent. High-risk queries are automatically routed to human experts, while standard queries are handled by the LLM with sub-second latency.

2. Architecture Diagram



3. Component Description

Component	Responsibility
Document Loader	Uses PyPDFLoader to extract raw text from customer policy documents.
Chunking Strategy	Breaks text into \$1000\$-character segments to ensure context fits within the LLM's attention span.
Embedding Model	all-MiniLM-L6-v2 converts text chunks into mathematical vectors.
Vector Store	ChromaDB stores and indexes vectors for fast semantic retrieval.
Retriever	Queries the Vector Store to find the top-\$K\$ most relevant document splits.
LLM	Llama 3.3 (via Groq) processes the retrieved context to generate a natural language response.
Workflow Engine	LangGraph maintains the "State" of the conversation and manages node transitions.
Routing Layer	A logic gate that inspects query intent to determine the execution path (AI vs. Human).
HITL Module	A blocking node that pauses the AI to allow for manual human intervention.

4. Data Flow

PDF to Vector (Ingestion)

1. **Load:** PDF is read into memory.
2. **Split:** Text is recursively split into overlapping chunks.
3. **Embed:** Chunks are passed through the embedding model.
4. **Index:** Vectors and metadata are stored in the persistent ChromaDB folder.

Query Lifecycle (Inference)

1. **Input:** User enters a query.

2. **Router:** The system checks for "High-Risk" keywords (e.g., *refund*, *lawsuit*).
3. **Standard Path:** If safe, the system retrieves relevant chunks → feeds them to the LLM → returns the answer.
4. **Escalation Path:** If risky, the system pauses → waits for human input → returns the human's answer.

5. Technology Choices

- **ChromaDB:** Chosen for its **local persistence** and zero-config setup. It allows the knowledge base to stay on-disk without needing a dedicated cloud database server.
- **LangGraph:** Chosen because it treats the LLM as an **agent**. Unlike linear chains, LangGraph can handle cycles, complex routing, and "Human-in-the-loop" states natively.
- **Groq (Llama 3.3):** Chosen for its **Inference Speed**. Using LPUs (Language Processing Units), Groq delivers responses almost instantly, which is critical for customer satisfaction.
- **HuggingFace Embeddings:** Provides high-quality, open-source models that run locally on CPU, reducing API dependency and costs.

6. Scalability Considerations

- **Handling Large Documents:** The system uses **Recursive Character Splitting**, which allows it to process a 1-page PDF or a 1000-page manual by indexing only the necessary segments into ChromaDB.
- **Increasing Query Load:** Since the vector store is local, query latency stays low (<50ms for retrieval). As load increases, the system can be containerized using Docker to scale horizontally.
- **Latency Concerns:** By using **Groq**, we mitigate the "LLM Bottleneck." Most of the processing time is spent on local retrieval rather than waiting for cloud-based generation.